



Capítulo 7 - Controle de sinais ferroviários

O sinal deve apresentar uma indicação coerente com as condições conhecidas pelo programa.

Introdução	Nos capítulos anteriores, comandamos desvios, lemos sensores e acompanhamos uma sequência de passagens. Agora vamos transformar condições lógicas em indicações luminosas de um sinal ferroviário.
Objetivo	Controlar um sinal didático de dois aspectos, vermelho e verde, utilizando funções e verificações antes de mudar a indicação.
Meta de aprendizagem	Ao final, você deverá conseguir acender os LEDs corretamente, iniciar com indicação de parada e impedir o verde quando uma condição necessária não estiver atendida.

1. O que o sinal representa?

O sinal ferroviário apresenta uma indicação ao operador. Neste exercício, utilizaremos um modelo simplificado de dois aspectos: vermelho para parada e verde para prosseguir quando as condições didáticas estiverem atendidas.

Aspecto é o conjunto de luzes apresentado pelo sinal. Sistemas ferroviários reais podem utilizar outros aspectos e regras próprias. O modelo deste capítulo foi escolhido para aprender a programação.

Acender o vermelho não corta a energia dos trilhos e não para automaticamente uma locomotiva. Neste capítulo, o programa controla somente a indicação luminosa.

2. Começar com indicação de parada

O programa iniciará com indicação vermelha. O verde somente poderá ser solicitado depois que as informações simuladas tiverem sido fornecidas e verificadas.

Também voltará ao vermelho quando houver bloqueio, perda de uma condição necessária ou solicitação manual de parada.

Durante a inicialização da placa, antes de `setup()`, e na falta de alimentação, os LEDs podem ficar apagados. O código, sozinho, não garante vermelho nesses momentos.

3. Condições para o exemplo

Vamos utilizar cinco variáveis lógicas. Elas serão modificadas pelo Monitor Serial para simular o sistema, sem movimentar trens ou desvios.



Variável	Informação representada
informationValid	As informações utilizadas para a decisão foram consideradas válidas.
sectionClear	O trecho protegido foi considerado livre.
routeReady	A rota foi considerada preparada.
emergencyActive	Existe uma condição de emergência.
proceedRequested	O operador solicitou a indicação de prosseguir.

O verde depende de todas as permissões e da ausência de emergência. Se qualquer condição necessária deixar de ser atendida, o resultado da decisão será vermelho.

Uma entrada Hall não acionada não prova que o trecho esteja livre. A passagem do ímã da locomotiva também não confirma a passagem do último vagão. Neste exercício, sectionClear é uma informação simulada pelo operador.

Da mesma forma, um comando enviado ao servo não confirma a posição física das agulhas. routeReady é uma condição simulada, e não uma confirmação automática dos desvios.

4. Montagem de bancada

Utilize dois LEDs separados, um vermelho e um verde. Cada LED deve ter seu próprio resistor em série. Para LEDs indicadores comuns, um resistor de 1 k Ω por LED é um ponto de partida conservador para este exercício em 3,3 V.

Elemento	Ligação de exemplo
LED vermelho	GPIO 25 $\rightarrow$ resistor de 1 k Ω $\rightarrow$ anodo do LED; catodo $\rightarrow$ GND.
LED verde	GPIO 26 $\rightarrow$ resistor de 1 k Ω $\rightarrow$ anodo do LED; catodo $\rightarrow$ GND.

O anodo é o terminal positivo do LED; o catodo é o terminal negativo. Confira a identificação dos terminais do componente. Nesta ligação, HIGH acende e LOW apaga.

Os GPIOs 25 e 26 são exemplos para uma ESP32 clássica baseada no ESP32-WROOM, com esses pinos disponíveis. Não representam o mapeamento definitivo de uma gaveta da ferrovia.

Monte com a alimentação desligada. Não conecte um LED diretamente ao GPIO sem resistor e não aplique 5 V à entrada ou saída da ESP32.

Um sinal comercial pode ter ligação de anodo comum, alimentação própria ou consumo maior. A ligação acima vale para os dois LEDs de bancada. Confira o circuito de um sinal pronto antes de conectá-lo; ele pode exigir um estágio de acionamento.



5. Separar decisão e acionamento

A função `canProceed()` decide se as condições permitem o verde. A função `setSignal()` aplica a indicação aos LEDs. A função `updateSignal()` liga essas duas tarefas.

```
bool canProceed() {  
    return informationValid && sectionClear && routeReady  
        && !emergencyActive && proceedRequested;  
}
```

Essa função retorna `true` somente quando todas as condições são atendidas. O operador `&&` significa E, e `!` inverte o valor lógico da emergência.

No acionamento, o código apaga as duas saídas antes de acender a luz escolhida. As instruções são executadas em sequência: existe um breve intervalo com as duas apagadas, mas não uma ordem de acender as duas ao mesmo tempo.

6. Comandos do exercício

Comando	Ação simulada
V / I	Informações válidas / informações inválidas.
L / O	Trecho livre / trecho ocupado.
A / B	Rota preparada / rota bloqueada.
E / C	Ativar / cancelar emergência.
G / R	Solicitar verde / solicitar vermelho.

Envie uma letra maiúscula por vez. Cancelar a emergência ou restaurar uma condição não recupera o verde automaticamente: é preciso fazer uma nova solicitação com G.

7. Programa completo

Crie o programa `intermediate_07_signals`. Com `simulationOnly = true`, o teste apresenta mensagens e não configura as saídas dos LEDs. Para o teste físico, use `false` depois de conferir a montagem.

O código está apresentado em blocos consecutivos. Copie todos, na ordem, para o mesmo programa.

```
const bool simulationOnly = true;  
const int redPin = 25;  
const int greenPin = 26;  
bool informationValid = false;  
bool sectionClear = false;  
bool routeReady = false;  
bool emergencyActive = false;  
bool proceedRequested = false;  
int lastAspect = -1;
```



```
bool canProceed() {
    return informationValid && sectionClear && routeReady
        && !emergencyActive && proceedRequested;
}

void setSignal(bool green) {
    int aspect = green ? 1 : 0;
    if (aspect == lastAspect) return;
    if (!simulationOnly) {
        digitalWrite(greenPin, LOW);
        digitalWrite(redPin, LOW);
        digitalWrite(green ? greenPin : redPin, HIGH);
    }
    lastAspect = aspect;
    Serial.println(green ? "Signal: GREEN" : "Signal: RED");
}

void updateSignal() {
    if (!informationValid || !sectionClear || !routeReady
        || emergencyActive) {
        proceedRequested = false;
    }
    setSignal(canProceed());
}

void setup() {
    if (!simulationOnly) {
        pinMode(redPin, OUTPUT);
        pinMode(greenPin, OUTPUT);
    }
    Serial.begin(115200);
    delay(1000);
    Serial.println("EFMN 74 - Two-aspect signal test");
    updateSignal();
    Serial.println("Commands: V I L O A B E C G R");
}

void loop() {
    if (Serial.available() > 0) {
        char command = Serial.read();
        switch (command) {
            case 'V': informationValid = true; break;
            case 'I': informationValid = false; break;
            case 'L': sectionClear = true; break;
            case 'O': sectionClear = false; break;
            case 'A': routeReady = true; break;
            case 'B': routeReady = false; break;
            case 'E': emergencyActive = true; break;
            case 'C': emergencyActive = false; break;
            case 'G': proceedRequested = true; break;
            case 'R': proceedRequested = false; break;
            default:
                if (command != '\n' && command != '\r') {
                    Serial.println("Invalid command");
                }
                break;
        }
        updateSignal();
    }
}
```



8. Entendendo o bloqueio da solicitação

updateSignal() apaga a solicitação de prosseguir quando as informações são inválidas, o trecho está ocupado, a rota está bloqueada ou há emergência.

Assim, se o sinal estava verde e o operador simula ocupação com O, o sinal vai para vermelho. Ao simular trecho livre com L, ele continua vermelho até receber uma nova solicitação com G.

lastAspect guarda a última indicação enviada. Dessa forma, o programa não repete a mensagem nem altera os LEDs quando o aspecto continua igual.

O exemplo atualiza a decisão após cada comando do Monitor Serial. Na integração futura, qualquer mudança de informação relevante deverá provocar nova avaliação, ou a função deverá ser chamada continuamente em loop().

Exercício prático - Simulação

Meta: conferir quando o verde é permitido e quando a indicação deve permanecer vermelha.

1. Mantenha simulationOnly = true e utilize uma ESP32 de bancada conectada por USB.
2. Selecione a placa e a porta, compile e envie o programa.
3. Abra o Monitor Serial em 115200 baud e pressione EN ou RESET.
4. Confira a mensagem Signal: RED.
5. Envie G antes de preparar as condições. O sinal deve continuar vermelho, sem uma nova mensagem de mudança.
6. Envie V, L e A, uma letra de cada vez. O sinal ainda deve permanecer vermelho.
7. Envie G novamente. Confira a mudança para verde.
8. Envie O. Confira o retorno imediato da decisão para vermelho.
9. Envie L. O sinal deve continuar vermelho.
10. Envie G. Confira a nova mudança para verde.
11. Envie E, depois C. O sinal deve ir para vermelho e permanecer vermelho após cancelar a emergência.
12. Envie G novamente e depois R. Confira verde e, em seguida, vermelho.

Durante uma solicitação impedida, nenhuma mensagem de mudança aparece se o sinal já estava vermelho. A ausência de nova mensagem não significa liberação.

Teste com LEDs

1. Desligue a alimentação e monte os LEDs com seus resistores.
2. Confira os GPIOs, as polaridades e o GND.
3. Troque simulationOnly para false, compile e envie.
4. Repita a sequência da simulação e compare os LEDs com as mensagens.



5. Confira que o vermelho apaga ao acender o verde e que o verde apaga ao voltar ao vermelho.

9. Integração com sensores e desvios

Para uma operação real, as variáveis simuladas precisam receber informações verificadas. A lógica de ocupação deve considerar o que cada sensor detecta, os limites dos trechos e a passagem da composição inteira.

A preparação da rota deve considerar a posição necessária de cada desvio, possíveis conflitos e a confiança disponível na informação de posição. Sem confirmação física, um comando de servo permanece uma posição comandada.

Uma mudança que torne as condições desconhecidas deve retirar a permissão de verde. Recuperar a informação ou cancelar uma emergência não deve, por si só, iniciar uma operação.

O programa deste capítulo não está ligado à chave de emergência da gaveta de controle principal, não comanda a tração e não verifica conflitos entre rotas. Ele é um exercício de lógica e indicação luminosa.

10. Quando o resultado não corresponder ao esperado

Comportamento	Verificação
Mensagens corretas, mas LEDs apagados.	Confira se a simulação foi desativada, os resistores, as polaridades, o GND e os GPIOs.
O verde não acende ao enviar G.	Confira V, L e A e se a emergência foi cancelada com C.
O sinal permanece vermelho após liberar o trecho.	Esse é o comportamento previsto: envie uma nova solicitação com G.
Os dois LEDs permanecem acesos.	Interrompa o teste e confira se há ligações cruzadas, outro tipo de sinal ou código diferente do exemplo.
O sinal fica apagado ao desligar a ESP32.	O circuito depende da alimentação; o código não mantém a indicação durante falta de energia.



Resumo do capítulo

- O exemplo utiliza vermelho para parada e verde para prosseguir.
- A indicação começa em vermelho quando o programa inicializa.
- O verde exige informações válidas, trecho livre, rota preparada, ausência de emergência e solicitação do operador.
- Uma condição de bloqueio retira a solicitação de prosseguir.
- Restaurar uma condição não recupera o verde automaticamente.
- Cada LED precisa de seu próprio resistor.
- Controlar a indicação não significa controlar a tração.
- Uma lógica de sinalização real exige verificações adicionais.

Próximo capítulo: Controle de passagens de nível.